

Web Development

>> PHP : Arrays, and Forms

Arrays

- An **array** in PHP is a special variable that can hold **multiple values** at the same time, under a single name. Arrays are used to store collections of related data — like a list of **names**, or product **prices**.
- Each value stored in an array is called an element.

```
$myArray = ['one', 2, '3'];  
$myArray[1] = 'two';      //Assign new value
```

- Types of Arrays in PHP:
 1. **Indexed Arrays**
 2. **Associative Arrays**
 3. **Multidimensional Arrays**

PHP provides built-in functions for array

PHP Array Functions

Function	Description
<u>array()</u>	Creates an array
<u>array_change_key_case()</u>	Changes all keys in an array to lowercase or uppercase
<u>array_chunk()</u>	Splits an array into chunks of arrays
<u>array_column()</u>	Returns the values from a single column in the input array
<u>array_combine()</u>	Creates an array by using the elements from one "keys" array and one "values" array
<u>array_count_values()</u>	Counts all the values of an array
<u>array_diff()</u>	Compare arrays, and returns the differences (compare values only)
<u>array_diff_assoc()</u>	Compare arrays, and returns the differences (compare keys and values)
<u>array_diff_key()</u>	Compare arrays, and returns the differences (compare keys only)
<u>array_diff_uassoc()</u>	Compare arrays, and returns the differences (compare keys and values, using a user-defined key comparison function)

lump.asp

https://www.w3schools.com/php/php_ref_array.asp

1- Indexed Arrays

```
<?php  
$fruits = array("Apple", "Banana", "Orange");  
echo $fruits[0]; // Output: Apple  
?>
```

- Alternative syntax (from PHP 5.4+):

```
$fruits = ["Apple", "Banana", "Orange"];
```

2- Associative Array

- Arrays with **named keys** (like a dictionary or map).

key => value

```
<?php
```

```
$person = array("name" => "Ali", "age" => 25, "city" =>  
"Cairo");
```

```
echo $person["name"];
```

```
// Output: Ali
```

```
?>
```

extract(\$arr)

```
<!DOCTYPE html>
<html>
<body>

<?php
$a = "Original";
$my_array = array("a" => "Cat", "b" => "Dog", "c" => "Horse");
extract($my_array);
echo "\$a = $a; \$b = $b; \$c = $c";
?>

</body>
</html>
```

\$a = Cat; \$b = Dog; \$c = Horse



in_array(item, \$arr)

```
<!DOCTYPE html>
<html>
<body>

<?php
$people = array("Peter", "Joe", "Glenn", "Cleveland");

if (in_array("Glenn", $people))
{
    echo "Match found";
}
else
{
    echo "Match not found";
}
?>

</body>
</html>
```

Match found

3- Multidimensional Arrays

- Arrays that contain other arrays.

```
<?php
```

```
$students = array(  
    array("Ali", 90, 85),  
    array("Sara", 92, 88),  
    array("Omar", 80, 75)  
);
```

```
echo $students[1][0]; // Output: Sara
```

```
?>
```

Looping Through Arrays

- Indexed Array (using for)

```
<?php
$colors = ["Red", "Green", "Blue"];
for ($i = 0; $i < count($colors); $i++) {
    echo $colors[$i] . "<br>";
}
```

- Associative Array (using foreach)

```
<?php
$user = ["username" => "ahmed", "email" => "ahmed@example.com"];
foreach ($user as $key => $value) {
    echo "$key: $value<br>";
}
```

Array Functions (Most Common)

- **count()** – Get number of elements

```
$count = count($fruits);
```

- **array_push()** – Add element(s) to end

```
array_push($fruits, "Mango");
```

- **array_pop()** – Remove last element

```
array_pop($fruits);
```

- **array_merge()** – Merge two arrays

```
$a = ["a", "b"];
```

```
$b = ["c", "d"];
```

```
$c = array_merge($a, $b); // ["a", "b", "c", "d"]
```

Array Functions (Most Common)

- **in_array()** – Check if value exists

```
if (in_array("Banana", $fruits)) {  
    echo "Found!";  
}
```

- **array_keys()** – Get all keys

```
$person = array("name" => "Ali", "age" => 25, "city" => "Cairo");  
$keys = array_keys($person); // ["name", "age", "city"]
```

- **sort()** – Sort indexed array ascending

```
sort($fruits);
```

- **asort()** – Sort associative array by value

```
asort($person);
```

- **ksort()** – Sort associative array by key

```
ksort($person);
```

PHP String Functions

- `strlen($str);`
- `str_word_count($str);`
- `strpos($str, $text);`
- `str_replace($replacethis, $replacewith, $str);`
- `str_repeat($str, $counter);` //repeat some str no. of times
- `substr($str, $start_pos, $length);` // length of substring
- `explode(separator, $str);` // break a string
- `implode(separator, $arr)` // join string using separator.

Forms in PHP

Form Submission

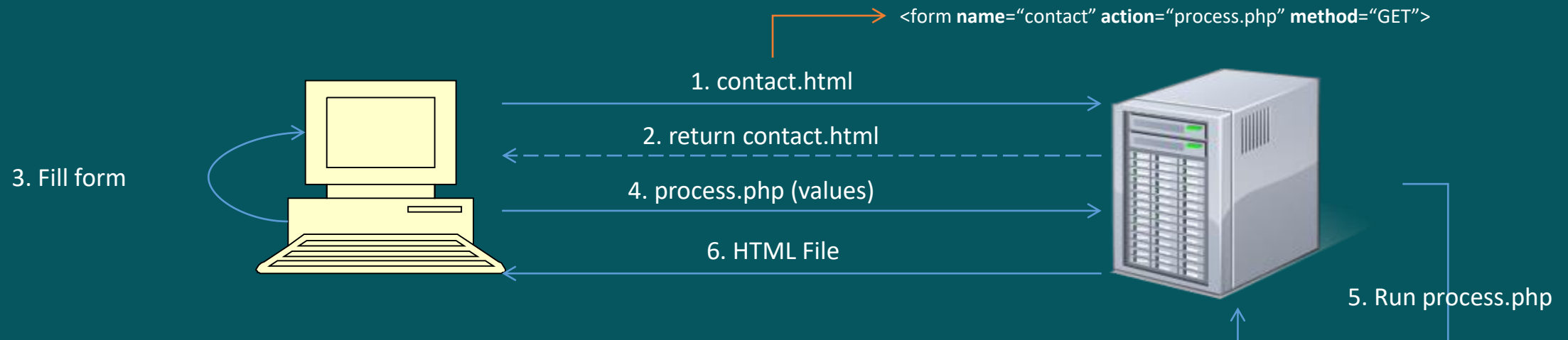
```
<form id=" " name=" " action=" " method=" " >  
</form>
```

For form identification

File/function to call

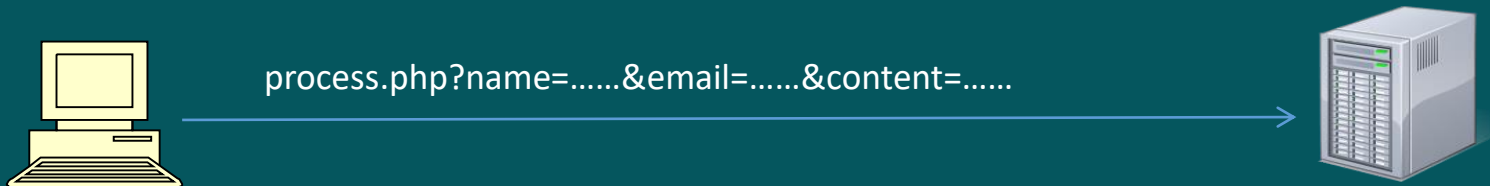
Method to submit the form

GET | POST



GET Method

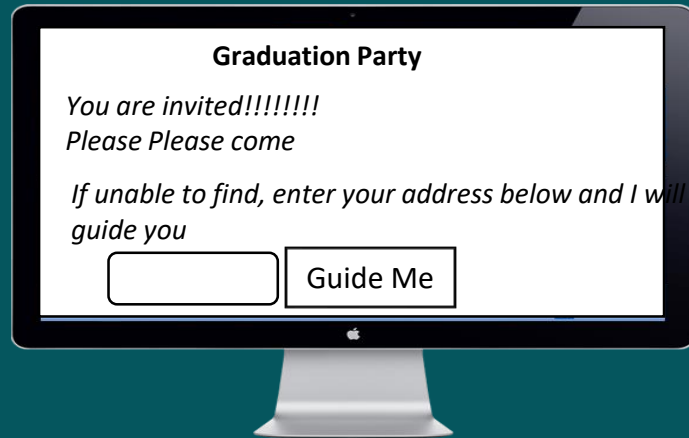
- Sends the form information by including it on the URL. Therefore, the get method is not secured to send sensitive information.
- The **GET** method is used to submit the HTML form data. This data is collected by the predefined **\$_GET variable** for processing.



When to use

- Short forms, with only 1 or 2 input fields
- Forms with select, radio, and checkbox fields

Google Maps



maps.google.com/maps?saddr=.....&daddr=.....

Maps with directions



```
<form action="http://maps.google.com/maps" method="get" target="_blank">  
  If unable to find, enter your address below and I will guide you  
  <input type="text" name="location" />  
  <input type="hidden" name="hiddenInfo" value="Never Giveup" />  
  <input type="submit" value="Guide Me" />  
</form>
```

Google Maps

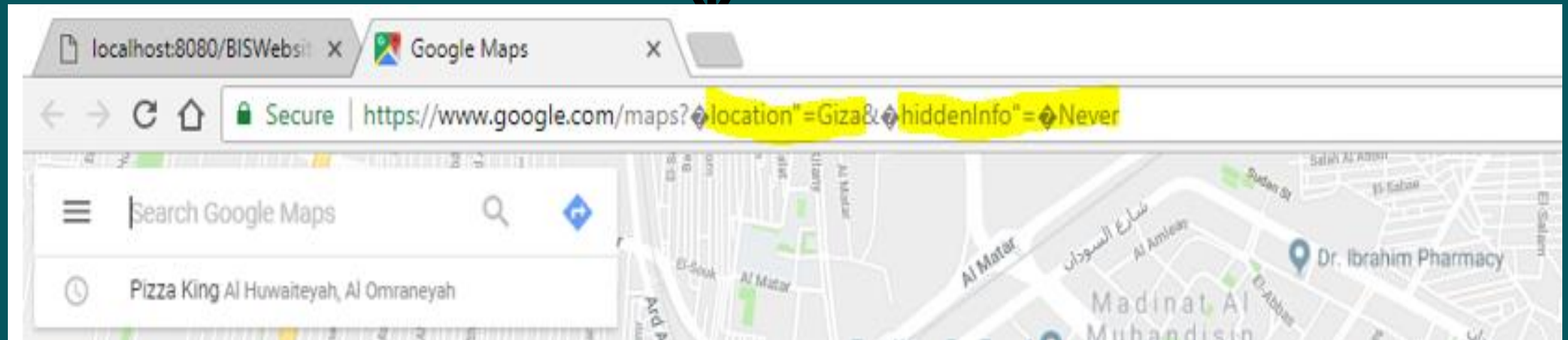
Graduation Party

You are invited!!!!!!!

Please Please come

If unable to find, enter your address below and I will guide you

Guide Me



POST Method

Sends the data to the server in two steps

1. Browser contacts the server
2. Send the information

the data submitted by this method is collected by the predefined super global variable **\$_POST**

When to use

- Longer forms, more than 3 input fields
- Forms with large textarea fields
- Forms where security is more important

Some Guidelines

- Always assign a **name** attribute to your form fields
 - Eg. `<input type="text" name="username"/>`
- Remember **Associative Arrays**

`name="username"` => *What the user enters*

Key Value

How will you access this information in PHP

`$uname = array-name [ 'username' ]`

`$_GET | $_POST`

1. Create a variable
2. Use the associative array technique
 - a. Index is the key name
3. What is the array-name?
 - a. Based on the method used

PHP Form Handling with GET

```
<form action="form-handler.php" method="GET">
  <fieldset>
    <legend>Personal information:</legend>
    <label for="fname" style="color: crimson">First name:</label><br />
    <input type="text" id="fname" name="fname" /> <br />

    <label for="email" style="color: crimson">E-mail:</label><br />
    <input type="text" id="email" name="email" /> <br />
    <input type="submit" />
  </fieldset>
</form>
```

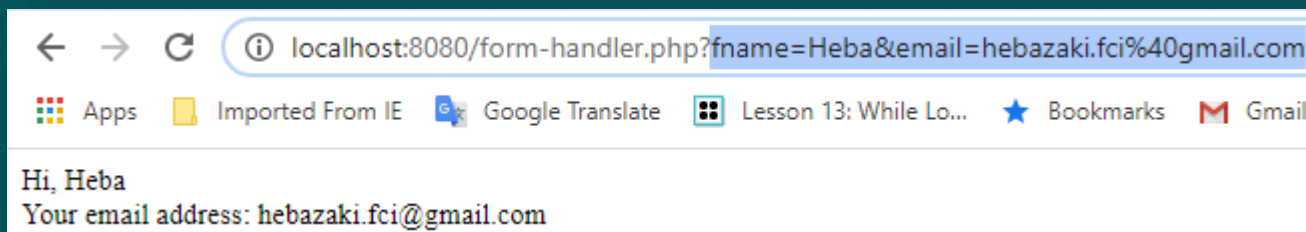
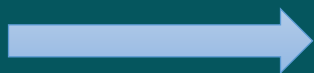
```
// getting the value of name field
$name = $_GET["fname"];
// getting the value of the email field
$email = $_GET["email"];

echo "Hi, ". $name . "<br>";
echo "Your email address: ". $email . "<br>";
```

Personal information:

First name:

E-mail:



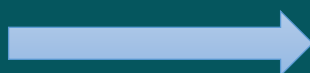
PHP Form Handling with POST

```
form-handler.php
1  <?php
2
3  // getting the value of name field
4  $name = $_POST["fname"];
5  // getting the value of the email field
6  $email = $_POST["email"];
7
8  echo "Hi, ". $name . "<br>";
9  echo "Your email address: ". $email . "<br>";
10
11 ?>
```

Personal information:

First name:

E-mail:



Hi, Heba
Your email address: hebazaki.fci@gmail.com

All Pre-defined Form Variables

- **\$_GET** variable can be used to collect form data sent with the GET method.

```
$varName = $_GET['username'];
```

- **\$_POST** variable can be used to collect form data sent with the POST method.

```
$varName = $_POST['username'];
```

- **\$_REQUEST** variable can be used to collect form data sent with both the GET and POST methods.

```
$varName = $_REQUEST['username'];
```

Using “name” attribute to get values of controls

```
About Me:<br />
<textarea name="aboutme"></textarea> <br />

Gender:
<input type="radio" name="gender" value="female" /> Female
<input type="radio" name="gender" value="male" /> Male
<br />
```

```
// getting the value of the aboutme field
$aboutme = $_POST["aboutme"];
// getting the value of the radio buttons
$gender = $_POST["gender"];
```

Personal information:

First name:

E-mail:

About Me:

Gender: ☒ Female ☐ Male

Hi, Sara
Your email address: ss.aa@hotmail.com
Your Details: I am FCI student, At Bioinformatics Department, Cairo University
Your gender: female

PHP Form Validation

- Check mandatory fields:

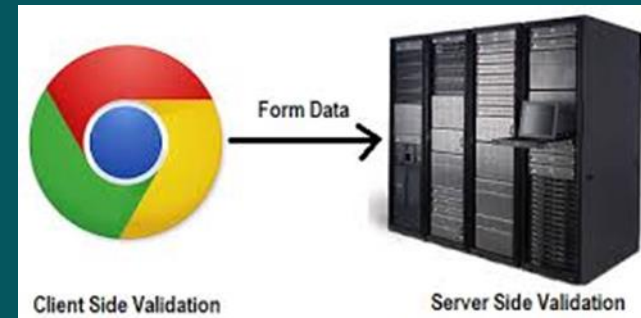
```
// getting the value of name field
$name = $_POST["name"];
// check if name is empty or not
if(empty($name)) {
    echo "Name is required";
}
```

- Check email validation:

```
// getting the value of the email field
$email = $_POST["email"];

// checking if the email value is valid
if(!filter_var($email, FILTER_VALIDATE_EMAIL))
{
    echo "Email value invalid";
}
```

- FILTER_SANITIZE_URL, FILTER_VALIDATE_IP



PHP WITH MYSQL

XAMPP Control Panel v3.3.0

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	19044 22196	80, 443	Stop Admin Config Logs
☐	MySQL	2940	3306	Stop Admin Config Logs
☐	FileZilla			Start Admin Config Logs
☐	Mercury			Start Admin Config Logs
☐	Tomcat			Start Admin Config Logs

Config Netstat Shell Explorer Services

10:52:50 PM [main] Checking for prerequisites
 10:52:55 PM [main] All prerequisites found
 10:52:55 PM [main] Initializing Modules
 10:52:55 PM [Apache] XAMPP Apache is already running
 10:52:55 PM [Apache] XAMPP Apache is already running
 10:52:55 PM [mysql] XAMPP MySQL is already running
 10:52:55 PM [main] Starting Check-Timer
 10:52:55 PM [main] Control Panel Ready

localhost/phpmyadmin/

phpMyAdmin

Recent Favorites

- New
- information_schema
- mysql
- performance_schema
- phpmyadmin
- test

Server: 127.0.0.1

Databases SQL Status User accounts Export Import Set

General settings

Server connection collation: utf8mb4_unicode_ci

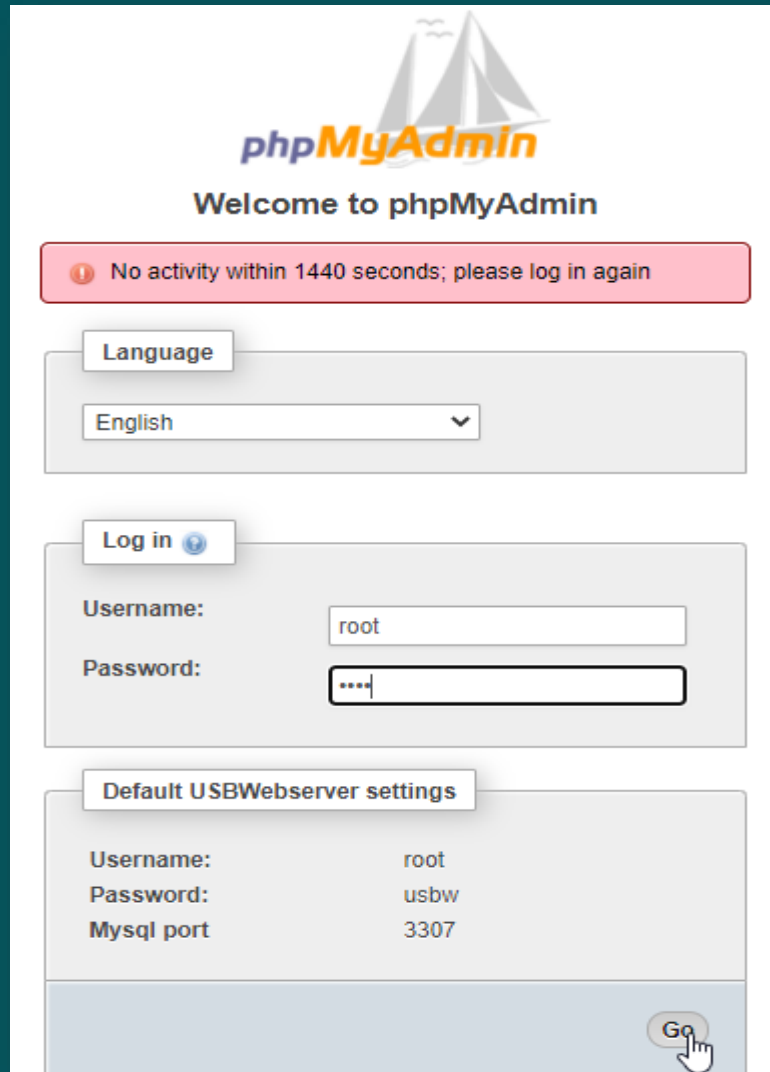
More settings

Appearance settings

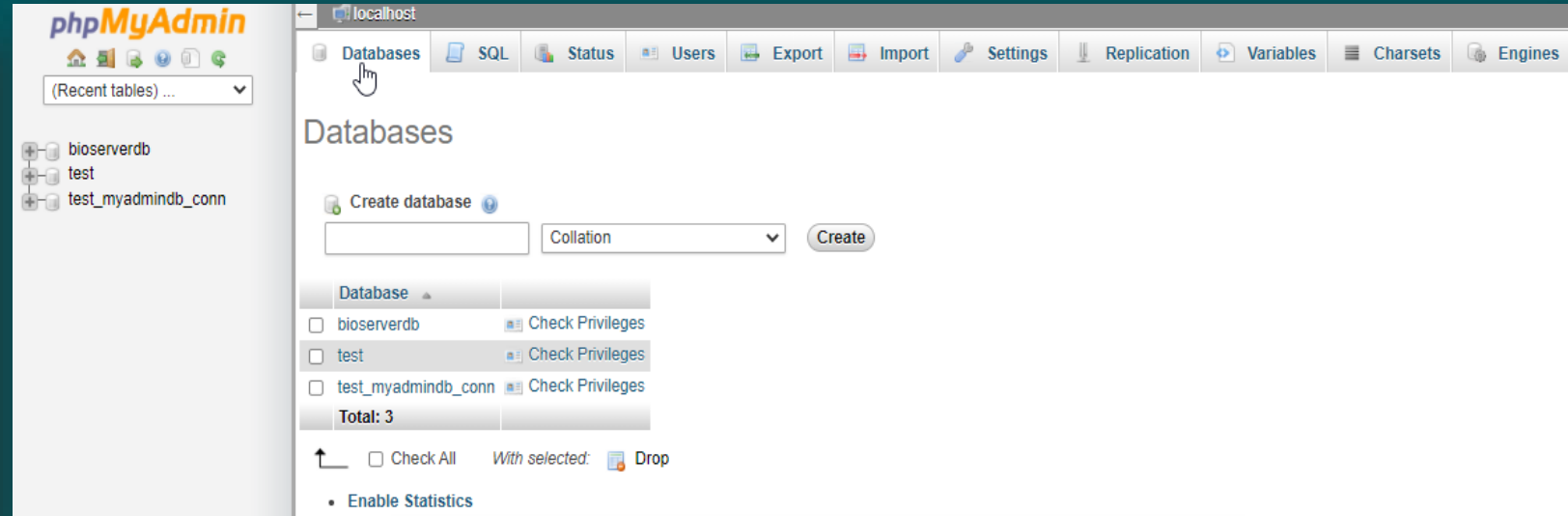
Language: English

Theme: pmahomme View all

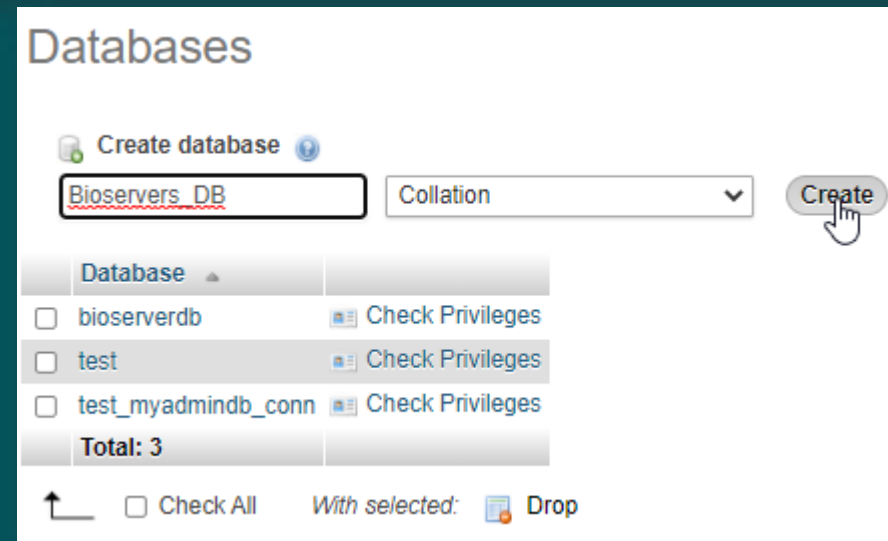
Php MyAdmin – Log in – Create DB



The image shows the phpMyAdmin welcome page. At the top is the phpMyAdmin logo and the text "Welcome to phpMyAdmin". Below this is a red message box that says "No activity within 1440 seconds; please log in again". There are three main sections: "Language" with a dropdown menu set to "English", "Log in" with fields for "Username:" (containing "root") and "Password:" (with masked characters), and "Default USBWebserver settings" with fields for "Username:" (root), "Password:" (usbw), and "Mysql port" (3307). A "Go" button is at the bottom right.



The image shows the phpMyAdmin interface for the "localhost" server. The left sidebar shows a tree view of databases: bioserverdb, test, and test_myadmindb_conn. The main panel is titled "Databases" and features a "Create database" form with a text input field and a "Collation" dropdown menu, followed by a "Create" button. Below this is a table listing existing databases: bioserverdb, test, and test_myadmindb_conn, each with a "Check Privileges" link. At the bottom, there are checkboxes for "Check All" and "Drop", and a link for "Enable Statistics".



The image shows a close-up of the "Create database" form in the phpMyAdmin interface. The text input field contains "Bioservers_DB" and is underlined. The "Collation" dropdown menu is open, showing a list of collations. The "Create" button is highlighted with a mouse cursor. Below the form is a table listing existing databases: bioserverdb, test, and test_myadmindb_conn, each with a "Check Privileges" link. At the bottom, there are checkboxes for "Check All" and "Drop", and a link for "Enable Statistics".

Php MyAdmin – Create table

phpMyAdmin

(Recent tables) ...

- bioserverdb
- bioservers_db
- test
- test_myadmindb_conn

localhost bioservers_db

Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers

No tables found in database

Create table

Name: Number of columns:

Go

Browse Structure SQL Search Insert Export Import Operations Triggers

Table name: Add column(s) Go

Structure

Name	Type	Length/Values	Default	Collation	Attributes	Null	Index	A.I	Comments
Gemone_ID	INT		None			☐	PRIMARY	☒	
GName	VARCHAR	2000	None			☐	---	☐	
RefSeq_No	INT		None			☐	---	☐	
CompleteGenome	VARCHAR		None			☐	---	☐	

Table comments:

Storage Engine:

Collation:

PARTITION definition:

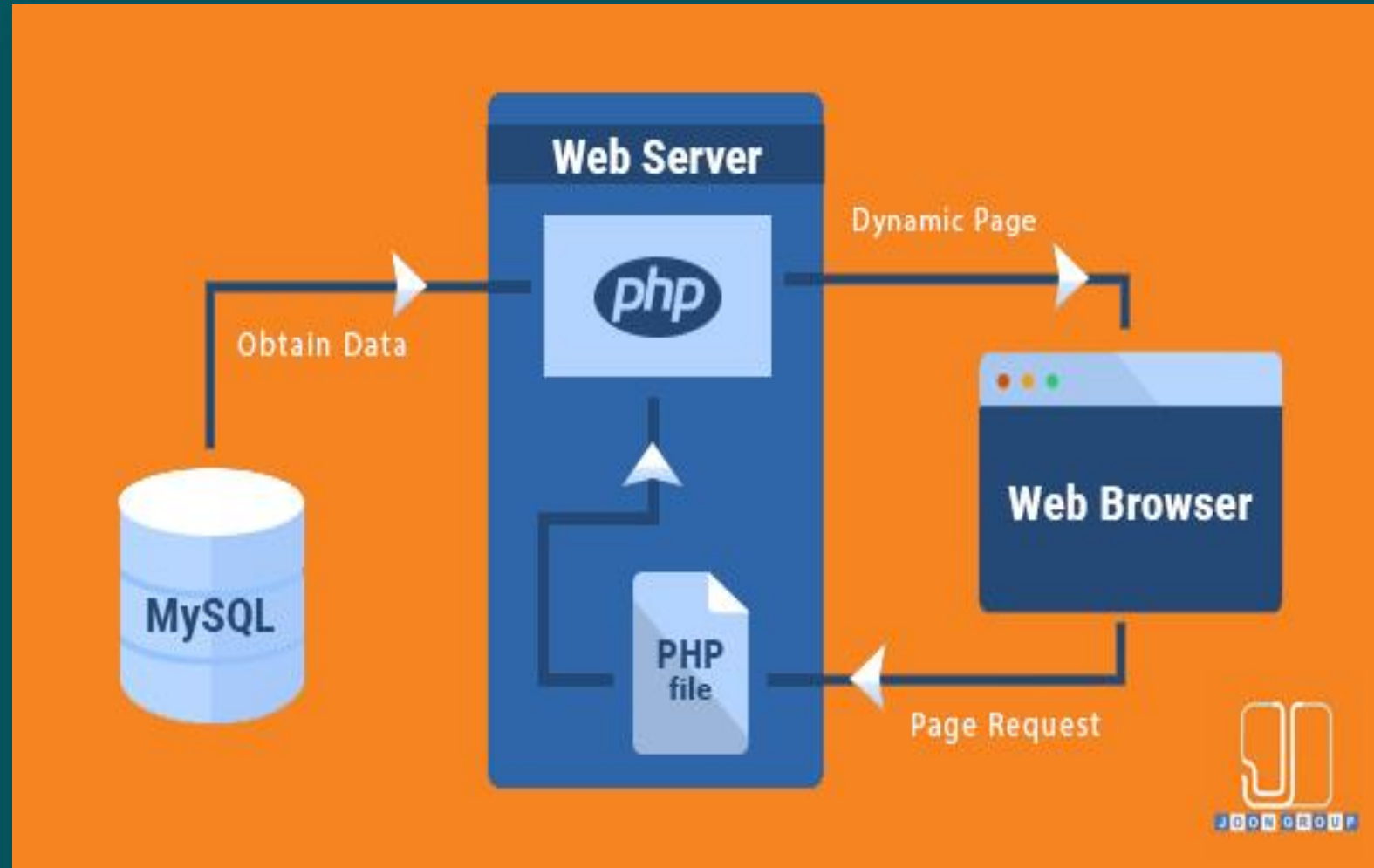
Save

Connect to MySQL Server using PHP

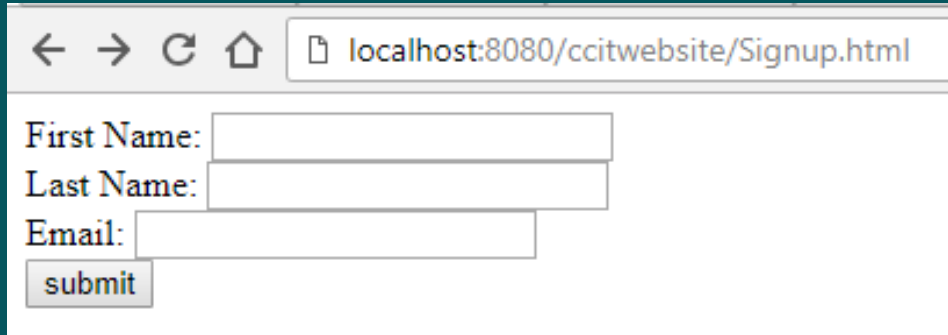
- PHP 5 and later can work with a MySQL database using either:
- **MySQLi extension** (the "i" stands for improved): only works with MySQL databases.
- **PDO** (PHP Data Objects): works on 12 different database systems.



How to Connect to MySQL Using PHP



Signup Form



A screenshot of a web browser window. The address bar shows 'localhost:8080/ccitwebsite/Signup.html'. The page content includes three text input fields labeled 'First Name:', 'Last Name:', and 'Email:'. Below the 'Email:' field is a 'submit' button.

1

Signup.html

```
<html>
```

```
<body>
```

```
<form action="Registration.php" method="post">
```

```
First Name: <input type="text" name="fname"/><br>
```

```
Last Name: <input type="text" name="lname"/><br>
```

```
Email: <input type="text" name="email"/><br>
```

```
<input type="submit" value="submit"/>
```

```
</form>
```

```
</body>
```

```
</html>
```

Signup Form

```
<?php
$serv="localhost";
$userN="root";
$pass="";
$dbname="MohSalah";

$conn=mysqli_connect($serv,$userN,$pass,$dbname);
if(!$conn){

    die("problem with the server".mysqli_connection_error);
}
```

2

Registration.php

Signup Form

```
$sql="INSERT INTO player (fn,ln,em) VALUES  
('$_POST[fname]','$_POST[lname]','$_POST[email])";
```

```
if(mysqli_query($conn,$sql))
```

```
    {header("location:index.php"); /* Redirect browser */  
    }
```

```
else {echo "An Error Occurred "};}
```

```
mysqli_close($conn);
```

```
?>
```

2

Registration.php

Login Form

```
<html>
<body>
    <form action="CheckLogin.php" method="post">
        User Name: <input type="text" name="un"/><br>
        Password : <input type="text" name="pass"/><br>
        <input type="submit" value="submit"/>
    </form>
</body>
</html>
```

1

LogIn.html

Login Form

```
<?php
$server="localhost";
$user="root";
$password="";
$dbname="bibbo";

$conn=mysqli_connect($server,$user,$password,$dbname);

if(!$conn){

    die("problem with the server".mysqli_connection_error);
}
```

2

CheckLogin.php

Login Form

```
else{

    $sql="SELECT * FROM CUST WHERE username= '$_POST[un]' AND
    password= '$_POST[pass]'";

    $res=mysqli_query($conn,$sql);

    if(mysqli_num_rows($res)==1)
    {
        header("location:Landpage.html");
    }
    else{
        echo "invalid user name or password";    }
    }
?>
```

2

CheckLogin.php

Data Retrieval

```
<?php
$serviceName="localhost";
$username="root";
$password="";
$db="bibbo";

$conn=mysqli_connect($serviceName,$username,$password,$db);
if(!$conn)
{die("Connection error".mysqli_connect_error());}
```

Data Retrieval

```
$sql="SELECT * FROM Cust";
$sqResult=mysqli_query($conn,$sql);
if(mysqli_num_rows($sqResult)>0){

    while($row=mysqli_fetch_assoc($sqResult)){

        echo "id" .$row["firstname"] ." last " .$row["lastname"]
        . "<br>";
    }
}
else {echo "0 records";}
mysqli_close($conn);
?>
```

Sessions

- An alternative way to make data accessible across the various pages of an entire website is to use a PHP Session. (Global Variables)

```
<?php
    session_start();

    if( isset( $_SESSION['counter'] ) ) {
        $_SESSION['counter'] += 1;
    }else {
        $_SESSION['counter'] = 1;
    }

    $msg = "You have visited this page ". $_SESSION['counter'];
    $msg .= "in this session.";
?>
```

```
<html>

    <head>
        <title>Setting up a PHP session</title>
    </head>

    <body>
        <?php echo ( $msg ); ?>
    </body>

</html>
```

Destroying a PHP Session

- All PHP session can be destroyed by **session_destroy()** function.
- Use **unset()** function (to destroy a single session variable).

```
<?php
    unset($_SESSION['counter']);
?>
```

```
<?php
    session_destroy();
?>
```

Thanks